

### **3.6 피마 인디언 당뇨병 예측**

## 목차

---

1. 실습 개요와 데이터 소개
  1. 피마 인디언 당뇨병 데이터란
  2. 컬럼 구성
  3. 라이브러리 임포트와 데이터 로딩
  4. `info()`로 데이터 타입 확인
2. 평가 함수 준비
  1. `get_clf_eval()` — 5개 지표 통합
  2. `precision_recall_curve_plot()`
3. 1차 모델링 — 전처리 없이 학습
  1. 피쳐/레이블 분리와 로지스틱 회귀
  2. 정밀도-재현율 곡선 확인
4. 데이터 클렌징 — 0값 문제 발견과 처리
  1. `describe()`로 이상 징후 포착
  2. Glucose 히스토그램
  3. 피쳐별 0값 건수 확인
  4. 0값을 평균으로 대체
5. 2차 모델링 — 전처리와 스케일링 적용
6. 임계값 조정으로 재현율 개선
  1. 임계값별 평가 함수
  2. 여러 임계값 비교
  3. 최종 임계값 0.48 적용
7. 핵심 요약 및 머신러닝 활용 포인트

## 1. 실습 개요와 데이터 소개

### 1.1 피마 인디언 당뇨병 데이터란

이번 장은 3.1~3.5장에서 배운 **평가 지표를 실전 데이터에 종합 적용**하는 실습입니다. **피마 인디언 당뇨병(Pima Indian Diabetes)** 데이터는 미국 애리조나주 원주민인 피마 인디언 부족의 건강 데이터로, **당뇨병 여부를 예측**하는 이진 분류 문제입니다.

피마 인디언은 1950년대까지 당뇨병 환자가 거의 없었으나, 미국 문화에 동화되면서 **비만과 당뇨 유병률이 급격히 증가**해 의학 연구의 주요 대상이 되었습니다.

#### ■ 이 실습에서 배우는 것

- 불균형 데이터에서 정확도만 보면 안 되는 이유의 실전 확인
- 결측치가 0으로 위장되어 있는 데이터의 발견과 처리
- 스케일링이 성능에 미치는 영향
- 임계값 조정으로 재현율을 끌어올리는 실무 기법

### 1.2 컬럼 구성

| 컬럼명                      | 의미                                 |
|--------------------------|------------------------------------|
| Pregnancies              | 임신 횟수                              |
| Glucose                  | 포도당 부하 검사 수치 (혈당)                  |
| BloodPressure            | 혈압 (mm Hg)                         |
| SkinThickness            | 팔 삼두근 뒤쪽 피하지방 측정값 (mm)             |
| Insulin                  | 혈청 인슐린 (mu U/ml)                   |
| BMI                      | 체질량지수 (체중kg / 키m <sup>2</sup> )    |
| DiabetesPedigreeFunction | 당뇨 가족력 수치                          |
| Age                      | 나이                                 |
| Outcome                  | 당뇨 여부 (0 = 음성, 1 = 양성) — 예측 대상 레이블 |

### 1.3 라이브러리 임포트와 데이터 로딩

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score
from sklearn.metrics import f1_score, confusion_matrix, precision_recall_curve, roc_curve
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

diabetes_data = pd.read_csv('diabetes.csv')
```

```
print(diabetes_data['Outcome'].value_counts())
diabetes_data.head(3)
```

| 줄   | 코드                                                            | 상세 설명                                                                       |
|-----|---------------------------------------------------------------|-----------------------------------------------------------------------------|
| 7~8 | from sklearn.metrics import<br>accuracy_score, ..., roc_curve | 3장에서 배운 <b>평가 지표</b> 를 <b>한꺼번에</b> 임포트합니다. 정확도부터 ROC 곡선까지 모두 포함되어 있습니다.     |
| 9   | from sklearn.preprocessing import<br>StandardScaler           | 2.5장에서 배운 <b>표준화 스케일러</b> 입니다. 뒤에서 성능 개선에 사용됩니다.                            |
| 10  | from sklearn.linear_model import<br>LogisticRegression        | 본 실습에서 사용할 <b>분류 알고리즘</b> 입니다. 로지스틱 회귀는 <b>스케일링에 민감</b> 하다는 특성이 뒤에서 중요해집니다. |
| 12  | diabetes_data = pd.read_csv('diabetes.csv')                   | CSV를 DataFrame으로 로딩합니다. 총 <b>768건 × 9컬럼</b> 입니다.                            |
| 13  | diabetes_data['Outcome'].value_counts()                       | 레이블 분포를 확인합니다. <b>음성 500건, 양성 268건</b> 으로 약 <b>65:35</b> 의 불균형입니다.          |

#### ▶ 실행 결과

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6     148             72             35         0   33.6              0.627    50         1
1            1      85             66             29         0   26.6              0.351    31         0
2            8     183             64              0         0   23.3              0.672    32         1
```

#### ⚠ head(3)에서 이미 이상 징후가 보입니다

상위 3건만 봐도 **Insulin**이 모두 0이고, 세 번째 행은 **SkinThickness**도 0입니다. 인슐린 수치나 피부 두께가 0이라는 것은 **의학적으로 불가능**합니다. 이는 측정하지 않은 값을 0으로 채워 넣은 것으로 의심됩니다. 4장에서 이를 본격적으로 다룹니다.

## 1.4 info( )로 데이터 타입 확인

```
diabetes_data.info( )
```

| 줄 | 코드                   | 상세 설명                                                          |
|---|----------------------|----------------------------------------------------------------|
| 1 | diabetes_data.info() | 컬럼별 <b>Non-Null</b> 건수와 데이터 타입을 확인합니다. 데이터 로딩 직후의 필수 점검 단계입니다. |

#### ▶ 실행 결과

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Pregnancies           768 non-null   int64
 1   Glucose                768 non-null   int64
 2   BloodPressure          768 non-null   int64
 3   SkinThickness          768 non-null   int64
 4   Insulin                768 non-null   int64
 5   BMI                   768 non-null   float64
 6   DiabetesPedigreeFunction 768 non-null   float64
```

```

7 Age                768 non-null    int64
8 Outcome            768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

```

### ■ 결과 해석 — 겉보기에는 완벽한 데이터

- 모든 컬럼이 **768 non-null**로 결측치가 하나도 없습니다.
- 모든 컬럼이 **int64** 또는 **float64**인 숫자형이라 인코딩도 필요 없습니다.

2.6장 타이타닉과 달리 전처리가 전혀 필요 없어 보입니다. 하지만 이것이 바로 이 실습의 함정입니다. `info()` 는 NaN만 찾아낼 뿐, 0으로 위장한 결측치는 발견하지 못합니다.

## 2. 평가 함수 준비

### 2.1 get\_clf\_eval() — 5개 지표 통합

```

# 수정된 get_clf_eval() 함수
def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}, \
F1: {3:.4f}, AUC: {4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))

```

| 줄     | 코드                                                                 | 상세 설명                                                                                   |
|-------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 2     | <code>def get_clf_eval(y_test, pred=None, pred_proba=None):</code> | <b>3.5장에서 완성한 최종 평가 함수</b> 를 그대로 가져왔습니다. 인자가 3개인 이유는 <b>ROC-AUC만 예측 확률이 필요하기</b> 때문입니다. |
| 3~7   | <code>confusion / accuracy / precision / recall / f1</code>        | 다섯 지표는 <b>예측 레이블(pred)</b> 로 계산합니다.                                                     |
| 9     | <code>roc_auc = roc_auc_score(y_test, pred_proba)</code>           | ROC-AUC만 <b>예측 확률(pred_proba)</b> 을 받습니다. 이 구분이 3장의 핵심 중 하나입니다.                         |
| 13~14 | <code>print('정확도: ... AUC: {4:.4f}')</code>                        | 5개 지표를 한 줄에 출력합니다. <code>\</code> 는 줄 연결 문자입니다.                                         |

### 2.2 precision\_recall\_curve\_plot()

```

def precision_recall_curve_plot(y_test=None, pred_proba_c1=None):
    # threshold ndarray와 이 threshold에 따른 정밀도, 재현율 ndarray 추출.
    precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_c1)

    # X축을 threshold값으로, Y축은 정밀도, 재현율 값으로 각각 Plot 수행. 정밀도는 점선으로 표시
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]

```

```
plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')
plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')

# threshold 값 X 축의 Scale을 0.1 단위로 변경
start, end = plt.xlim()
plt.xticks(np.round(np.arange(start, end, 0.1), 2))

# x축, y축 label과 legend, 그리고 grid 설정
plt.xlabel('Threshold value'); plt.ylabel('Precision and Recall value')
plt.legend(); plt.grid()
plt.show()
```

| 줄     | 코드                                                                | 상세 설명                                                               |
|-------|-------------------------------------------------------------------|---------------------------------------------------------------------|
| 1     | def precision_recall_curve_plot(y_test=None, pred_proba_c1=None): | 3.3장에서 만든 <b>정밀도-재현율 곡선 시각화 함수</b> 를 재사용합니다.                        |
| 7     | threshold_boundary = thresholds.shape[0]                          | precisions/recalls가 thresholds보다 <b>1개 길기 때문에</b> 길이를 맞추기 위한 처리입니다. |
| 8~9   | plt.plot(...)                                                     | 정밀도는 <b>점선</b> , 재현율은 <b>실선</b> 으로 그립니다.                            |
| 12~13 | start, end = plt.xlim() / plt.xticks(...)                         | x축 눈금을 <b>0.1 간격</b> 으로 재설정해 임계값을 읽기 쉽게 만듭니다.                       |

## 3. 1차 모델링 — 전처리 없이 학습

### 3.1 피쳐/레이블 분리와 로지스틱 회귀

```
# 피쳐 데이터 세트 X, 레이블 데이터 세트 y를 추출.
# 맨 끝이 Outcome 컬럼으로 레이블 값임. 컬럼 위치 -1을 이용해 추출
X = diabetes_data.iloc[:, :-1]
y = diabetes_data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 156, stratify=y)

# 로지스틱 회귀로 학습, 예측 및 평가 수행.
lr_clf = LogisticRegression(solver='liblinear')
lr_clf.fit(X_train, y_train)
pred = lr_clf.predict(X_test)
pred_proba = lr_clf.predict_proba(X_test)[:, 1]

get_clf_eval(y_test, pred, pred_proba)
```

| 줄   | 코드                                                             | 상세 설명                                                                                                                                                |
|-----|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3~4 | X = diabetes_data.iloc[:, :-1] / y = diabetes_data.iloc[:, -1] | <b>맨 마지막 컬럼(Outcome)을 제외한 전부를</b> 피쳐로, <b>마지막 컬럼</b> 을 레이블로 분리합니다. 2.5장에서 배운 관용구입니다.                                                                 |
| 6   | train_test_split(..., stratify=y)                              | <b>stratify=y</b> 가 핵심입니다. 레이블의 클래스 비율(65:35)을 <b>학습·테스트 세트에 동일하게 유지</b> 하며 분할합니다. 불균형 데이터에서는 거의 필수 옵션입니다. 768건 → 학습 <b>614건</b> / 테스트 <b>154건</b> . |
| 9   | LogisticRegression(solver='liblinear')                         | 작은 데이터에 적합한 solver를 지정해 <b>수렴 경고를 방지</b> 합니다.                                                                                                        |
| 11  | pred = lr_clf.predict(X_test)                                  | 예측 레이블(0/1)을 얻습니다.                                                                                                                                   |

|    |                                                               |                                             |
|----|---------------------------------------------------------------|---------------------------------------------|
| 12 | <code>pred_proba = lr_clf.predict_proba(X_test)[: , 1]</code> | 클래스 1(당뇨 양성)일 확률만 추출합니다. ROC-AUC 계산에 필요합니다. |
| 14 | <code>get_clf_eval(y_test, pred, pred_proba)</code>           | 5개 지표를 한꺼번에 확인합니다.                          |

## ▶ 실행 결과

```
오차 행렬
[[87 13]
 [22 32]]
정확도: 0.7727, 정밀도: 0.7111, 재현율: 0.5926, F1: 0.6465, AUC:0.8083
```

## ⚠ 결과 해석 — 재현율이 매우 낮다

|           | 예측 0 (정상) | 예측 1 (당뇨) |
|-----------|-----------|-----------|
| 실제 0 (정상) | TN = 87   | FP = 13   |
| 실제 1 (당뇨) | FN = 22   | TP = 32   |

정확도는 77.27%로 그럭저럭 보이지만, **재현율이 0.5926에 불과합니다.** 이는 실제 당뇨 환자 54명 중 22명을 놓쳤다는 뜻입니다.

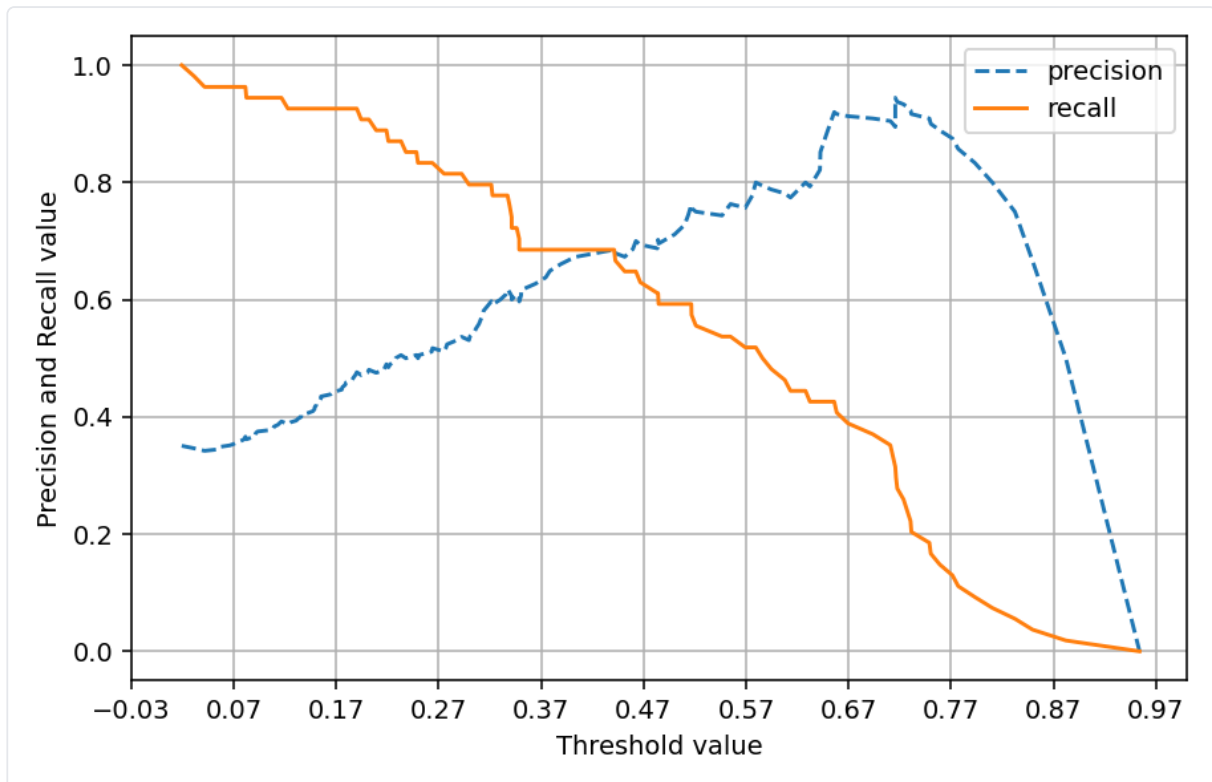
질병 진단에서 FN(환자를 정상으로 오판)은 가장 위험한 오류입니다. 치료 시기를 놓치기 때문입니다. 따라서 이 모델은 실무에 쓸 수 없는 수준이며, 재현율을 반드시 개선해야 합니다.

## 3.2 정밀도-재현율 곡선 확인

```
pred_proba_c1 = lr_clf.predict_proba(X_test)[: , 1]
precision_recall_curve_plot(y_test, pred_proba_c1)
```

| 줄 | 코드                                                               | 상세 설명                       |
|---|------------------------------------------------------------------|-----------------------------|
| 1 | <code>pred_proba_c1 = lr_clf.predict_proba(X_test)[: , 1]</code> | Positive 클래스의 예측 확률을 추출합니다. |
| 2 | <code>precision_recall_curve_plot(y_test, pred_proba_c1)</code>  | 임계값 조정 여지를 시각적으로 파악합니다.     |

## ▶ 실행 결과



전처리 전 모델의 정밀도(점선)-재현율(실선) 곡선

#### ■ 그래프 해석

임계값 **0.42** 부근에서 정밀도와 재현율이 균형을 이룹니다. 재현율을 높이려면 임계값을 낮춰야 하는데, **0.4 이하에서는 정밀도가 급격히 하락**합니다. 즉 임계값 조정만으로는 개선에 한계가 있습니다.

따라서 임계값을 건드리기 전에 **데이터 자체를 개선**해야 합니다. 다음 장에서 그 작업을 합니다.

## 4. 데이터 클렌징 — 0값 문제 발견과 처리

### 4.1 describe()로 이상 징후 포착

```
diabetes_data.describe()
```

| 줄 | 코드                       | 상세 설명                                                                                              |
|---|--------------------------|----------------------------------------------------------------------------------------------------|
| 1 | diabetes_data.describe() | 숫자형 컬럼의 <b>기술 통계량</b> 을 확인합니다. <code>info()</code> 가 놓친 문제를 찾기 위한 단계로, <b>min 값에 특히 주목</b> 해야 합니다. |

#### ▶ 실행 결과

|       | Pregnancies | Glucose    | BloodPressure | SkinThickness | Insulin    | BMI        | DiabetesPedigreeFunction | Age        | Outcome    |
|-------|-------------|------------|---------------|---------------|------------|------------|--------------------------|------------|------------|
| count | 768.000000  | 768.000000 | 768.000000    | 768.000000    | 768.000000 | 768.000000 | 768.000000               | 768.000000 | 768.000000 |
| mean  | 3.845052    | 120.894531 | 69.105469     | 20.536458     | 79.799479  | 31.992578  | 0.471876                 | 33.240885  | 0.348968   |
| std   | 3.369578    | 31.972618  | 19.355807     | 15.952218     | 115.244002 | 7.884160   | 0.331329                 | 11.760232  | 0.476941   |
| min   | 0.000000    | 0.000000   | 0.000000      | 0.000000      | 0.000000   | 0.000000   | 0.078000                 | 21.000000  | 0.000000   |
| 25%   | 1.000000    | 99.000000  | 62.000000     | 0.000000      | 0.000000   | 27.300000  | 0.243750                 | 24.000000  | 0.000000   |
| 50%   | 3.000000    | 117.000000 | 72.000000     | 23.000000     | 30.500000  | 32.000000  | 0.372500                 | 29.000000  | 0.000000   |

|     |           |            |            |           |            |           |          |           |          |
|-----|-----------|------------|------------|-----------|------------|-----------|----------|-----------|----------|
| 75% | 6.000000  | 140.250000 | 80.000000  | 32.000000 | 127.250000 | 36.600000 | 0.626250 | 41.000000 | 1.000000 |
| max | 17.000000 | 199.000000 | 122.000000 | 99.000000 | 846.000000 | 67.100000 | 2.420000 | 81.000000 | 1.000000 |

### △ 결과 해석 — min 값에서 드러나는 문제

- **Glucose(혈당) min = 0** : 혈당이 0이면 사망입니다. 있을 수 없는 값입니다.
- **BloodPressure(혈압) min = 0** : 혈압 0도 마찬가지로 불가능합니다.
- **SkinThickness min = 0, BMI min = 0** : 신체 측정값이 0일 수 없습니다.
- **Insulin min = 0**이고 **25%와 50% 지점도 0에 가깝습니다**(25%가 0). 절반 가까이 0이라는 뜻입니다.

반면 **Pregnancies(임신 횟수) min = 0**은 정상입니다. 임신한 적 없는 경우가 실제로 존재하기 때문입니다.

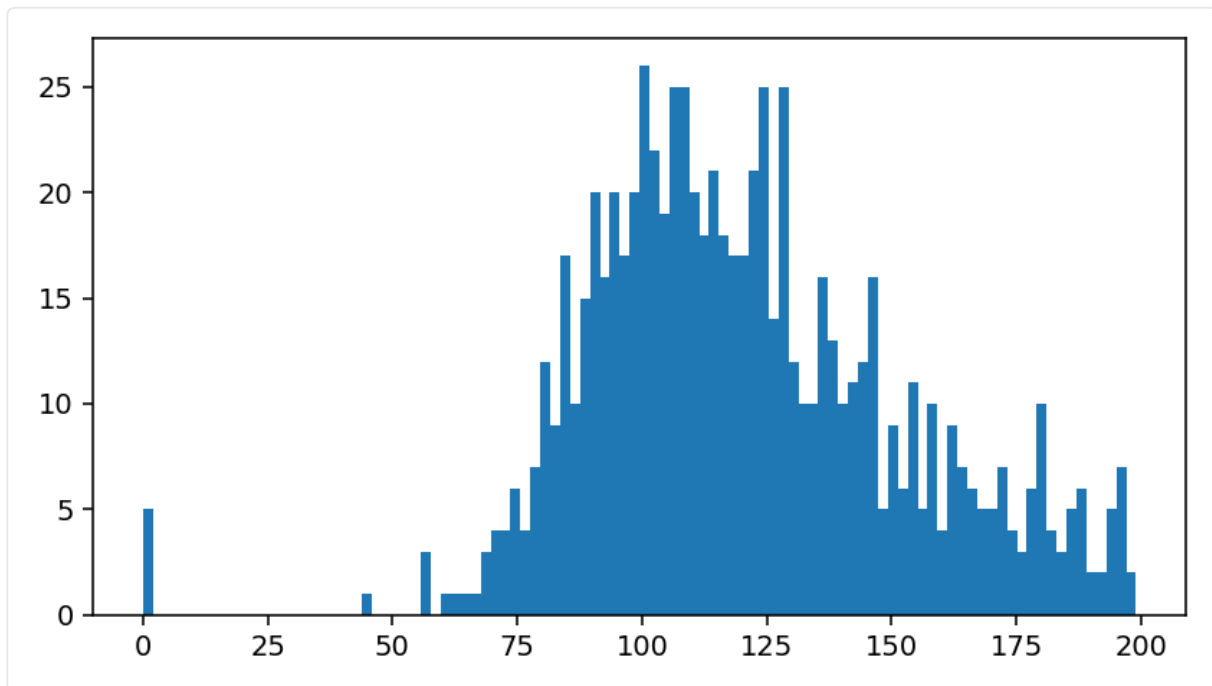
결론: 이 데이터의 0은 결측치가 위장된 것입니다. `info()`로는 절대 찾을 수 없었던 문제를 `describe()`가 드러냈습니다.

## 4.2 Glucose 히스토그램

```
plt.hist(diabetes_data['Glucose'], bins=100)
plt.show()
```

| 줄 | 코드                                                        | 상세 설명                                                                                           |
|---|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1 | <code>plt.hist(diabetes_data['Glucose'], bins=100)</code> | 히스토그램으로 값의 분포를 시각화합니다. <code>bins=100</code> 은 구간을 100개로 나누는 뜻으로, 잘게 나눌수록 <b>세밀한 분포</b> 가 보입니다. |

### ▶ 실행 결과



Glucose(혈당) 분포 — 왼쪽 끝 0 부근의 막대가 이상치

### ■ 그래프 해석

대부분의 값이 100~150 사이에 정규 분포에 가깝게 몰려 있는데, 0 부근에 작은 막대가 따로 떨어져 있습니다. 이 고립된 막대가 바로 0으로 채워진 결측치입니다. 정상적인 분포에서 벗어난 값이 눈으로 확인됩니다.

### 4.3 피처별 0값 건수 확인

```
# 0값을 검사할 피처명 리스트 객체 설정
zero_features = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

# 전체 데이터 건수
total_count = diabetes_data['Glucose'].count()

# 피처별로 반복 하면서 데이터 값이 0 인 데이터 건수 추출하고, 퍼센트 계산
for feature in zero_features:
    zero_count = diabetes_data[diabetes_data[feature] == 0][feature].count()
    print('{0} 0 건수는 {1}, 퍼센트는 {2:.2f} %'.format(feature, zero_count, 100*zero_count/total_count))
```

| 줄  | 코드                                                                              | 상세 설명                                                                                                 |
|----|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 2  | zero_features = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI'] | 0이 <b>의학적으로 불가능한 5개 피처</b> 만 골라 리스트로 만듭니다. <b>Pregnancies</b> 는 <b>의도적으로 제외</b> 했습니다 — 0이 정상이기 때문입니다. |
| 5  | total_count = diabetes_data['Glucose'].count()                                  | 전체 건수 768을 얻습니다. 퍼센트 계산의 분모입니다.                                                                       |
| 9  | zero_count = diabetes_data[diabetes_data[feature] == 0][feature].count()        | <b>불린 인덱싱</b> (1.4장)으로 해당 피처가 0인 행만 필터링한 뒤 건수를 셉니다. 대괄호가 두 번 쓰인 것은 <b>행 필터링 → 컬럼 선택</b> 순서입니다.        |
| 10 | print('{0} 0 건수는 {1}, 퍼센트는 {2:.2f} %'...)                                       | 건수와 비율을 함께 출력해 <b>심각도를 판단</b> 할 수 있게 합니다.                                                             |

#### ▶ 실행 결과

```
Glucose 0 건수는 5, 퍼센트는 0.65 %
BloodPressure 0 건수는 35, 퍼센트는 4.56 %
SkinThickness 0 건수는 227, 퍼센트는 29.56 %
Insulin 0 건수는 374, 퍼센트는 48.70 %
BMI 0 건수는 11, 퍼센트는 1.43 %
```

#### ■ 결과 해석 — 피처별 심각도가 크게 다르다

| 피처                   | 0 건수 | 비율            | 판단                    |
|----------------------|------|---------------|-----------------------|
| Glucose              | 5    | 0.65%         | 미미함 — 대체해도 영향 적음      |
| BMI                  | 11   | 1.43%         | 미미함                   |
| BloodPressure        | 35   | 4.56%         | 적은 편                  |
| <b>SkinThickness</b> | 227  | <b>29.56%</b> | <b>심각 — 3건 중 1건</b>   |
| <b>Insulin</b>       | 374  | <b>48.70%</b> | <b>매우 심각 — 절반이 결측</b> |

**Insulin**은 거의 **절반이 결측**입니다. 이 정도면 평균 대체 시 **데이터의 절반이 동일한 값**이 되어 피처로서의 변별력이 크게 떨어 집니다. 실무에서는 **컬럼 자체를 삭제**하거나 더 정교한 대체 기법(KNN Imputer, 회귀 대체 등)을 고려할 수준입니다.

## 4.4 0값을 평균으로 대체

```
# zero_features 리스트 내부에 저장된 개별 피처들에 대해서 0값을 평균 값으로 대체
mean_zero_features = diabetes_data[zero_features].mean()
diabetes_data[zero_features]=diabetes_data[zero_features].replace(0, mean_zero_features)
```

| 줄 | 코드                                                             | 상세 설명                                                                                                                            |
|---|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 2 | mean_zero_features =<br>diabetes_data[zero_features].mean()    | 5개 피처의 <b>평균값을 Series</b> 로 계산합니다. 인덱스가 컬럼명, 값이 평균인 형태입니다.                                                                       |
| 3 | diabetes_data[zero_features].replace(0,<br>mean_zero_features) | <b>replace(찾을값, 바꿀값)</b> 으로 <b>0을 평균으로 치환</b> 합니다. 두 번째 인자가 <b>Series</b> 이므로 <b>컬럼별로 각자의 평균</b> 이 적용됩니다. 결과를 다시 대입해야 원본에 반영됩니다. |

### ▶ 계산된 평균값

```
Glucose      120.894531
BloodPressure 69.105469
SkinThickness 20.536458
Insulin       79.799479
BMI           31.992578
dtype: float64
```

### ⚠ 이 방식의 통계적 한계

평균을 **0을 포함한 채로 계산**했다는 점에 주의해야 합니다. Insulin의 경우 절반이 0이므로 **실제 평균보다 훨씬 낮게 계산된 값** (79.8)으로 대체하게 됩니다. 0을 제외한 실제 평균은 약 155로, 두 배 가까이 차이가 납니다.

더 정확하게 하려면 **0을 NaN으로 먼저 바꾼 뒤** 평균을 구해야 합니다.

```
import numpy as np
diabetes_data[zero_features] = diabetes_data[zero_features].replace(0, np.nan)
diabetes_data[zero_features] = diabetes_data[zero_features].fillna(
    diabetes_data[zero_features].mean())
```

교재는 설명의 단순화를 위해 위 방식을 택했으며, 본 문서는 **교재 코드 기준의 실행 결과**를 싣습니다.

## 5. 2차 모델링 — 전처리와 스케일링 적용

```
X = diabetes_data.iloc[:, :-1]
y = diabetes_data.iloc[:, -1]

# StandardScaler 클래스를 이용해 피처 데이터 세트에 일괄적으로 스케일링 적용
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2, random_state = 156, stratify=y)

# 로지스틱 회귀로 학습, 예측 및 평가 수행.
lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)
pred = lr_clf.predict(X_test)
pred_proba = lr_clf.predict_proba(X_test)[: , 1]
```

```
get_clf_eval(y_test , pred, pred_proba)
```

| 줄   | 코드                                                             | 상세 설명                                                                                                                                                                             |
|-----|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1~2 | X = diabetes_data.iloc[:, :-1] 등                               | <b>0값이 대체된 데이터</b> 로 피쳐와 레이블을 다시 분리합니다.                                                                                                                                           |
| 5~6 | scaler = StandardScaler() / X_scaled = scaler.fit_transform(X) | <b>표준화</b> 를 적용해 모든 피쳐를 평균 0, 분산 1로 맞춥니다. 이 데이터는 Insulin(0~846)과 DiabetesPedigreeFunction(0.078~2.42)처럼 <b>스케일 차이가 극심</b> 해서 스케일링 효과가 큼니다. 로지스틱 회귀는 <b>거리·가중치 기반</b> 이라 특히 민감합니다. |
| 8   | train_test_split(X_scaled, y, ...)                             | 스케일링된 데이터로 분할합니다. <b>random_state</b> 와 <b>stratify</b> 는 1차와 동일해 <b>공정한 비교</b> 가 가능합니다.                                                                                          |
| 11  | lr_clf = LogisticRegression()                                  | 이번에는 <b>solver</b> 를 지정하지 않았습니다. 스케일링을 했으므로 기본 solver( <b>lbfgs</b> )로도 <b>수렴 경고 없이</b> 잘 학습됩니다. 스케일링의 부수적 효과입니다.                                                                 |
| 16  | get_clf_eval(y_test, pred, pred_proba)                         | 1차 모델과 <b>동일한 지표</b> 로 평가해 개선 정도를 확인합니다.                                                                                                                                          |

#### ▶ 실행 결과

```
오차 행렬
[[90 10]
 [21 33]]
정확도: 0.7987, 정밀도: 0.7674, 재현율: 0.6111, F1: 0.6804, AUC:0.8433
```

#### ■ 결과 해석 — 전처리의 효과

| 지표         | 1차 (전처리 전) | 2차 (전처리 후)    | 변화             |
|------------|------------|---------------|----------------|
| 정확도        | 0.7727     | <b>0.7987</b> | +0.0260        |
| 정밀도        | 0.7111     | <b>0.7674</b> | +0.0563        |
| 재현율        | 0.5926     | <b>0.6111</b> | +0.0185        |
| F1         | 0.6465     | <b>0.6804</b> | +0.0339        |
| <b>AUC</b> | 0.8083     | <b>0.8433</b> | <b>+0.0350</b> |

모든 지표가 개선되었습니다. 특히 **AUC가 0.8083 → 0.8433**으로 크게 올랐는데, AUC는 임계값과 무관한 지표이므로 이는 **모델의 근본적인 판별 능력이 향상**되었음을 의미합니다.

다만 **재현율은 여전히 0.6111**로 낮습니다. 실제 당뇨 환자 54명 중 **21명을 여전히 놓치고** 있습니다. 데이터 개선만으로는 부족하므로, 다음 단계에서 **임계값을 조정**합니다.

## 6. 임계값 조정으로 재현율 개선

### 6.1 임계값별 평가 함수

```
from sklearn.preprocessing import Binarizer

def get_eval_by_threshold(y_test , pred_proba_c1, thresholds):
    # thresholds 리스트 객체내의 값을 차례로 iteration하면서 Evaluation 수행.
```

```
for custom_threshold in thresholds:
    binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_c1)
    custom_predict = binarizer.transform(pred_proba_c1)
    print('임계값:', custom_threshold)
    get_clf_eval(y_test, custom_predict, pred_proba_c1)
```

| 줄   | 코드                                                            | 상세 설명                                                                              |
|-----|---------------------------------------------------------------|------------------------------------------------------------------------------------|
| 3   | def get_eval_by_threshold(y_test, pred_proba_c1, thresholds): | 3.3장의 함수와 거의 같지만, <b>마지막 줄에서 pred_proba_c1</b> 을 추가로 전달하는 점이 다릅니다. AUC 계산을 위해서입니다. |
| 6~7 | Binarizer(threshold=custom_threshold).fit(...).transform(...) | 지정한 임계값으로 확률을 <b>0/1 레이블로 변환</b> 합니다.                                              |
| 9   | get_clf_eval(y_test, custom_predict, pred_proba_c1)           | 변환된 레이블과 <b>원본 확률</b> 을 함께 넘깁니다. <b>AUC는 임계값과 무관하므로 항상 같은 값</b> 이 나옵니다.            |

## 6.2 여러 임계값 비교

```
thresholds = [0.3, 0.33, 0.36, 0.39, 0.42, 0.45, 0.48, 0.50]
pred_proba = lr_clf.predict_proba(X_test)
get_eval_by_threshold(y_test, pred_proba[:,1].reshape(-1,1), thresholds)
```

| 줄 | 코드                                                           | 상세 설명                                                                                                          |
|---|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1 | thresholds = [0.3, 0.33, 0.36, 0.39, 0.42, 0.45, 0.48, 0.50] | <b>0.5 이하의 값만</b> 나열했습니다. 목표가 <b>재현율 개선</b> 이므로 임계값을 낮추는 방향만 탐색하는 것입니다. 간격도 0.03으로 촘촘하게 잡아 <b>세밀하게 탐색</b> 합니다. |
| 3 | pred_proba[:,1].reshape(-1,1)                                | Positive 확률만 뽑아 <b>2차원으로 변환</b> 합니다. Binarizer가 2차원만 받기 때문입니다.                                                 |

### ▶ 실행 결과

```
임계값: 0.3
오차 행렬
[[67 33]
 [11 43]]
정확도: 0.7143, 정밀도: 0.5658, 재현율: 0.7963, F1: 0.6615, AUC:0.8433
임계값: 0.33
오차 행렬
[[72 28]
 [12 42]]
정확도: 0.7403, 정밀도: 0.6000, 재현율: 0.7778, F1: 0.6774, AUC:0.8433
임계값: 0.36
오차 행렬
[[76 24]
 [15 39]]
정확도: 0.7468, 정밀도: 0.6190, 재현율: 0.7222, F1: 0.6667, AUC:0.8433
임계값: 0.39
오차 행렬
[[78 22]
 [16 38]]
정확도: 0.7532, 정밀도: 0.6333, 재현율: 0.7037, F1: 0.6667, AUC:0.8433
임계값: 0.42
오차 행렬
[[84 16]
 [18 36]]
정확도: 0.7792, 정밀도: 0.6923, 재현율: 0.6667, F1: 0.6792, AUC:0.8433
임계값: 0.45
```

```

오차 행렬
[[85 15]
 [18 36]]
정확도: 0.7857, 정밀도: 0.7059, 재현율: 0.6667, F1: 0.6857, AUC:0.8433
임계값: 0.48
오차 행렬
[[88 12]
 [19 35]]
정확도: 0.7987, 정밀도: 0.7447, 재현율: 0.6481, F1: 0.6931, AUC:0.8433
임계값: 0.5
오차 행렬
[[90 10]
 [21 33]]
정확도: 0.7987, 정밀도: 0.7674, 재현율: 0.6111, F1: 0.6804, AUC:0.8433

```

### ■ 결과 해석 — 트레이드오프 정밀 관찰

| 임계값 | 0.30          | 0.33   | 0.36   | 0.39   | 0.42   | 0.45   | 0.48          | 0.50          |
|-----|---------------|--------|--------|--------|--------|--------|---------------|---------------|
| 정확도 | 0.7143        | 0.7403 | 0.7468 | 0.7532 | 0.7792 | 0.7857 | <b>0.7987</b> | <b>0.7987</b> |
| 정밀도 | 0.5658        | 0.6000 | 0.6190 | 0.6333 | 0.6923 | 0.7059 | 0.7447        | <b>0.7674</b> |
| 재현율 | <b>0.7963</b> | 0.7778 | 0.7222 | 0.7037 | 0.6667 | 0.6667 | 0.6481        | 0.6111        |
| F1  | 0.6615        | 0.6774 | 0.6667 | 0.6667 | 0.6792 | 0.6857 | <b>0.6931</b> | 0.6804        |

- AUC는 모든 임계값에서 **0.8433**으로 동일합니다. 임계값은 판정 기준을 옮길 뿐 모델의 판별력 자체를 바꾸지 않는다는 사실을 명확히 보여 줍니다.
- 임계값 **0.3**에서 **재현율 0.7963**으로 가장 높지만, **정밀도가 0.5658로 급락**합니다. 당뇨로 판정한 사람의 43%가 오진이라는 뜻입니다.
- **0.48**에서 **F1이 0.6931로 최대**가 됩니다. 정밀도와 재현율의 균형이 가장 좋은 지점입니다.
- 정확도는 0.48과 0.50이 같지만, **재현율은 0.48이 더 높습니다**(0.6481 vs 0.6111).

## 6.3 최종 임계값 0.48 적용

```

# 임계값을 0.48로 설정한 Binarizer 생성
binarizer = Binarizer(threshold=0.48)

# 위에서 구한 lr_clf의 predict_proba() 예측 확률 array에서 1에 해당하는 컬럼값을 Binarizer변환.
pred_th_048 = binarizer.fit_transform(pred_proba[:, 1].reshape(-1,1))

get_clf_eval(y_test , pred_th_048, pred_proba[:, 1])

```

| 줄 | 코드                                                                          | 상세 설명                                                                                   |
|---|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 2 | binarizer = Binarizer(threshold=0.48)                                       | 앞의 비교에서 <b>F1이 가장 높았던 0.48</b> 을 최종 임계값으로 선택합니다.                                        |
| 5 | pred_th_048 =<br>binarizer.fit_transform(pred_proba[:,<br>1].reshape(-1,1)) | <b>fit</b> 과 <b>transform</b> 을 <b>한 번에</b> 수행합니다. Binarizer는 fit에서 학습하는 것이 없으므로 안전합니다. |
| 7 | get_clf_eval(y_test, pred_th_048,<br>pred_proba[:, 1])                      | 최종 성능을 확인합니다. AUC 인자로는 <b>변환 전 원본 확률</b> 을 넘겨야 합니다.                                     |

### ▶ 실행 결과

오차 행렬

[[88 12]

[19 35]]

정확도: 0.7987, 정밀도: 0.7447, 재현율: 0.6481, F1: 0.6931, AUC:0.8433

## ■ 최종 결과 — 전체 개선 과정 요약

| 단계                   | 정확도    | 정밀도    | 재현율           | F1            | AUC           |
|----------------------|--------|--------|---------------|---------------|---------------|
| ① 전처리 전 (임계값 0.5)    | 0.7727 | 0.7111 | 0.5926        | 0.6465        | 0.8083        |
| ② 0값 대체 + 스케일링       | 0.7987 | 0.7674 | 0.6111        | 0.6804        | <b>0.8433</b> |
| ③ 임계값 <b>0.48</b> 적용 | 0.7987 | 0.7447 | <b>0.6481</b> | <b>0.6931</b> | 0.8433        |

최종적으로 재현율이 0.5926 → 0.6481로 약 5.6%p 개선되었습니다. 놓치는 환자가 22명에서 19명으로 3명 줄었습니다. 정밀도는 0.7674 → 0.7447로 소폭 하락했지만, 질병 진단에서는 합리적인 교환입니다.

## ⚠ 여전히 남은 과제

재현율 0.6481은 실제 당뇨 환자의 35%를 여전히 놓친다는 뜻입니다. 실제 의료 현장이라면 이 수준으로는 부족합니다. 추가로 시도할 수 있는 개선 방향은 다음과 같습니다.

- 임계값을 더 낮추기 : 0.3으로 하면 재현율 0.7963까지 오릅니다. 정밀도 하락을 감수할지는 의료적 판단의 영역입니다.
- 더 강력한 알고리즘 : 4장의 랜덤 포레스트, XGBoost, LightGBM 적용
- 결측치 처리 개선 : 0을 NaN으로 바꾼 뒤 평균 계산, 또는 KNN Imputer 사용
- Insulin 컬럼 재검토 : 48.7%가 결측이므로 삭제도 고려
- 클래스 불균형 대응 : `class_weight='balanced'` 옵션이나 SMOTE 오버샘플링

## 7. 핵심 요약 및 머신러닝 활용 포인트

## 7.1 실습 흐름 정리

| 단계 | 작업                             | 핵심 발견 / 결과                                        |
|----|--------------------------------|---------------------------------------------------|
| ①  | 데이터 로딩· <code>info()</code> 확인 | 결측치 0건, 전부 숫자형 — 문제없어 보임                          |
| ②  | 1차 모델링                         | 재현율 <b>0.5926</b> — 환자 22명 놓침                     |
| ③  | <code>describe()</code> 확인     | <b>min=0인 불가능한 값 발견</b>                           |
| ④  | 히스토그램                          | 0 부근의 <b>고립된 막대</b> 시각적 확인                        |
| ⑤  | 0값 건수 집계                       | Insulin <b>48.7%</b> , SkinThickness <b>29.6%</b> |
| ⑥  | 0값 평균 대체 + 스케일링                | AUC <b>0.8083 → 0.8433</b>                        |
| ⑦  | 임계값 탐색                         | 0.48에서 <b>F1 최대</b>                               |
| ⑧  | 최종 적용                          | 재현율 <b>0.6481</b> 로 개선                            |

## 7.2 핵심 API 요약표

| API                                       | 역할          | 주의 사항                                                     |
|-------------------------------------------|-------------|-----------------------------------------------------------|
| <code>df.describe()</code>                | 기술 통계량      | <b>min/max로 이상값 탐지</b> — <code>info()</code> 가 못 찾는 문제 발견 |
| <code>plt.hist(data, bins=n)</code>       | 분포 시각화      | <code>bins</code> 를 크게 하면 세밀한 분포 확인                       |
| <code>df[df[col]==0][col].count()</code>  | 조건 만족 건수    | 불린 인덱싱 활용                                                 |
| <code>df.replace(0, Series)</code>        | 값 치환        | Series 전달 시 <b>컬럼별 다른 값 적용</b>                            |
| <code>StandardScaler()</code>             | 표준화         | 로지스틱 회귀·SVM·KNN에 <b>필수</b>                                |
| <code>train_test_split(stratify=y)</code> | 계층적 분할      | <b>불균형 데이터에 필수</b>                                        |
| <code>Binarizer(threshold=t)</code>       | 임계값 기준 변환   | <b>2차원 입력 필요</b>                                          |
| <code>get_clf_eval(y, pred, proba)</code> | 5개 지표 통합 평가 | AUC만 <b>확률</b> 사용                                         |

## 7.3 머신러닝 활용 포인트

- **info()**만 믿지 말 것 : 결측치가 NaN이 아니라 **0, -1, 999, 'N/A' 같은 값으로 위장되어** 있는 경우가 실무에 매우 흔합니다. `describe()`의 min/max와 히스토그램을 반드시 확인하세요.
- **도메인 지식이 필수** : "혈압이 0일 수 없다"는 판단은 **통계가 아니라 의학 상식**에서 나옵니다. Pregnancies의 0은 정상이라는 구분도 마찬가지입니다. **데이터가 무엇을 의미하는지 이해**하는 것이 전처리의 출발점입니다.
- **결측 비율에 따라 전략을 달리** : 1~5%면 평균 대체로 충분하지만, **30~50%면 컬럼 삭제나 정교한 대체 기법**을 고려해야 합니다.
- **AUC로 진짜 개선을 확인** : 임계값을 바꾸면 정확도·정밀도·재현율은 다 움직이지만 AUC는 그대로입니다. **AUC가 올랐다면 모델이 실제로 좋아진 것**입니다.
- **임계값은 마지막 조정 수단** : 데이터 품질 개선과 모델 개선을 먼저 하고, 임계값은 **업무 요구사항에 맞춰 미세 조정**하는 데 사용하세요.
- **업무 맥락이 지표를 결정** : 질병 진단은 재현율, 스팸 필터는 정밀도가 우선입니다. **F1이 최대인 지점이 항상 정답은 아닙니다.**

### ⚠ 안정화 버전 참고

- `Binarizer`는 **기준값 초과일 때만 1을 반환**합니다(같으면 0). 경계값 처리에 주의하세요.
- `LogisticRegression()`의 기본 solver는 `'lbfgs'`입니다. **스케일링하지 않은 데이터**에서는 `ConvergenceWarning`이 발생할 수 있으므로 `solver='liblinear'`를 지정하거나 스케일링을 적용하세요.
- pandas 2.x부터 `value_counts()` 출력에 `Name: count`가 함께 표시됩니다.
- 결측치를 더 정교하게 대체하려면 사이킷런의 `SimpleImputer(strategy='mean')`나 `KNNImputer`를 사용할 수 있습니다. 이들은 `Pipeline`에 포함시켜 **데이터 누수를 방지**할 수 있다는 장점도 있습니다.